LAPORAN TUGAS KULIAH PENGOLAHAN CITRA DIGITAL
MATERI KONVOLUSI

SYAFIQ SYADIDUL AZMI

G6401231075

PCD K3

ILMU KOMPUTER
SEKOLAH SAINS DATA MATEMATIKA DAN INFORMATIKA
INSTITUT PERTANIAN BOGOR
BOGOR
2026

DAFTAR ISI

I TUGAS 1 1
1.1 Tugas 1.1 - Deteksi Tepi Daun (Manual) 1
1.1.1 Perhitungan Korelasi pada Piksel (2,2) 1
1.1.2 Perhitungan Konvolusi pada Piksel (2,2) 1
1.1.3 Analisis Perbandingan 2
1.2 Tugas 1.2 – Smoothing Citra Buah (Manual) 2
1.2.1 Filter Mean (Box Filter) 3×3 2
1.2.2 Filter Median 2
1.2.3 Filter Max 3
1.2.4 Analisis Efektivitas terhadap Salt Noise 3
II TUGAS 2 4
2.1 Tugas 2.1 – Eksplorasi Filter pada Citra Daun 4
2.1.1 Persiapan Dataset dan Preprocessing 4
2.1.2 Definisi Kernel 4
2.1.3 Hasil Visualisasi 5
2.1.4 Analisis 7
2.1.4.1 Fitur yang Ditonjolkan Masing-Masing Filter 8
2.1.4.2 Gaussian untuk Reduksi Noise pada Citra Daun 8
2.1.4.3 Laplacian/Sobel untuk Pengukuran Luas Daun dan Deteksi Lesi 8
2.1.4.4 Filter Paling Berguna untuk Sistem Deteksi Dini Penyakit 8
2.2 Tugas 2.2 – Perbandingan Penanganan Batas (Border Handling) 9
2.2.1 Pembuatan Citra Sintetis 9
2.2.2 Implementasi dan Hasil Numerik 9
2.2.3 Analisis 10
2.2.3.1 Rekomendasi untuk Aplikasi Pertanian 10
2.3 Tugas 2.3 – Aplikasi Sederhana: Identifikasi Tekstur Penyakit 11
2.3.1 (a) Desain Kernel 11
2.3.2 (b) Hasil Rata-Rata Output Konvolusi per Kelas 11
2.3.3 (c) Evaluasi Kernel 12
2.3.4 (d) Diskusi: Konvolusi Fixed Kernel vs. Learned Kernels pada CNN 12

DAFTAR TABEL

Tabel 1 Daftar citra yang digunakan pada Tugas 2.1 4
Tabel 2 Empat metode penanganan batas yang diimplementasikan 9
Tabel 3 Rata-rata respons kernel Laplacian 4-tetangga per kelas 11

DAFTAR GAMBAR

Gambar 1 Hasil konvolusi pada citra *apple-scab* 5
Gambar 2 Hasil konvolusi pada citra *grape-blackrot* 6
Gambar 3 Hasil konvolusi pada citra *peach-bacterialspot* 6
Gambar 4 Hasil konvolusi pada citra *soybean-healthy* 7
Gambar 5 Hasil konvolusi pada citra *tomato-leafmold* 7
Gambar 6 Perbandingan keempat metode penanganan batas pada citra sintetis 6×6 10
Gambar 7 Bar chart perbandingan rata-rata |output konvolusi| ketiga kelas penyakit 12

I TUGAS 1

1.1 Tugas 1.1 - Deteksi Tepi Daun (Manual)

$$I = \begin{bmatrix} 10 & 20 & 30 & 200 & 210 \\ 15 & 25 & 35 & 205 & 215 \\ 12 & 22 & 32 & 195 & 205 \\ 8 & 18 & 28 & 190 & 200 \\ 5 & 15 & 25 & 185 & 195 \end{bmatrix}$$

Matrix di atas merupakan Matriks citra digital dengan ukuran 5x5 piksel. Setiap elemen dalam matriks tersebut mewakili intensitas piksel pada posisi tertentu dalam citra. Tujuan tugas ini adalah

1.1.1 Perhitungan Korelasi pada Piksel (2,2)

Pada operasi korelasi, kernel G_x digunakan secara langsung tanpa rotasi. Proses perhitungan dilakukan dengan mengalikan elemen kernel dengan area 3×3 di sekitar piksel $(2, 2)$ pada matriks I:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Langkah perhitungan "multiply and accumulate" adalah sebagai berikut:

$$\begin{aligned} S &= (-1 \times 25) + (0 \times 35) + (1 \times 205) \\ &\quad +(-2 \times 22) + (0 \times 32) + (2 \times 195) \\ &\quad +(-1 \times 18) + (0 \times 28) + (1 \times 190) \\ S &= (-25 + 0 + 205) + (-44 + 0 + 390) + (-18 + 0 + 190) \\ &= (180) + (346) + (172) \\ &= 698 \end{aligned}$$

Jadi, nilai piksel pada posisi $(2, 2)$ setelah dilakukan korelasi adalah **698**.

1.1.2 Perhitungan Konvolusi pada Piksel (2,2)

Pada operasi konvolusi, kernel G_x harus dirotasi terlebih dahulu sebesar 180° sebelum dikalikan dengan citra. Kernel Sobel vertikal setelah dirotasi menjadi:

$$G_{x'} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Karena kernel ini tidak simetris terhadap rotasi 180°, maka hasil konvolusi akan berbeda dengan hasil korelasi. Jika dilakukan perhitungan "multiply and accumulate" pada posisi $(2, 2)$ menggunakan kernel yang telah dirotasi:

$$
\begin{aligned}
S &= (1 \times 25) + (0 \times 35) + (-1 \times 205) \\
&\quad +(2 \times 22) + (0 \times 32) + (-2 \times 195) \\
&\quad +(1 \times 18) + (0 \times 28) + (-1 \times 190) \\
&= -698
\end{aligned}
$$

Jadi, nilai piksel $(2, 2)$ hasil konvolusi adalah **−698**. Perbedaan tanda ini menunjukkan arah gradien yang dideteksi oleh kernel.

1.1.3 Analisis Perbandingan

a. **Alasan Perbedaan**: Perbedaan tanda ($+$ dan $-$) disebabkan oleh rotasi 180° pada operasi konvolusi terhadap kernel yang asimetris.
b. **Pemilihan Metode**: Korelasi lebih tepat karena memberikan nilai positif pada transisi gelap-ke-terang, sesuai dengan struktur citra daun yang diamati karena matriks I menunjukkan peningkatan intensitas dari kiri ke kanan, yang sesuai dengan deteksi tepi vertikal oleh kernel Sobel.

1.2 Tugas 1.2 – Smoothing Citra Buah (Manual)

Diberikan citra A berukuran 3×3 dengan **salt noise** pada pusatnya (255):

$$
A = \begin{bmatrix} 34 & 35 & 36 \\ 37 & 255 & 39 \\ 40 & 41 & 42 \end{bmatrix}
$$

1.2.1 Filter Mean (Box Filter) 3×3

Filter mean menghitung rata-rata dari seluruh piksel di dalam jendela kernel. Kernel w:

$$
w = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}
$$

Perhitungan pada posisi pusat (1,1):

$$
\begin{aligned}
S &= \frac{1}{9}(34 + 35 + 36 + 37 + 255 + 39 + 40 + 41 + 42) \\
&= \frac{1}{9}(559) \\
&\approx 62.11
\end{aligned}
$$

1.2.2 Filter Median

Filter median mengambil nilai tengah setelah seluruh piksel di dalam jendela diurutkan.

a. **Urutkan nilai neighborhood**: $34, 35, 36, 37, 39, 40, 41, 42, 255$
b. **Ambil nilai tengah (posisi ke-5)**: Nilai median adalah **39**.

1.2.3 Filter Max

Filter max mengambil nilai tertinggi di dalam jendela kernel.

a. **Himpunan nilai**: $\{34, 35, 36, 37, 255, 39, 40, 41, 42\}$
b. **Nilai tertinggi**: **255**.

1.2.4 Analisis Efektivitas terhadap Salt Noise

1. **Filter Paling Efektif**: Filter Median.
2. **Alasan**:
   - **Salt noise** (nilai 255) adalah pencilan (**outlier**) yang sangat ekstrim.
   - **Filter Mean** gagal karena nilai 255 tetap ikut dijumlahkan, sehingga hasil akhirnya (62.11) tetap jauh lebih tinggi dari nilai asli piksel sekitarnya (sekitar 30-an), yang mengakibatkan citra terlihat agak kabur (**blur**).
   - **Filter Max** justru mempertahankan **noise** tersebut karena 255 adalah nilai maksimum.
   - **Filter Median** sangat efektif karena nilai ekstrim (255) akan selalu berada di ujung urutan, sehingga yang terpilih sebagai nilai baru adalah nilai yang lebih merepresentasikan lingkungan sekitarnya (39) tanpa mengaburkan tepi citra secara signifikan.

II TUGAS 2

2.1 Tugas 2.1 – Eksplorasi Filter pada Citra Daun

2.1.1 Persiapan Dataset dan Preprocessing

Dataset yang digunakan adalah **New Plant Disease Dataset** dari Kaggle. Dipilih 5 citra dari folder `train` yang mewakili kelas dengan karakteristik tekstur berbeda:

Tabel 1: Daftar citra yang digunakan pada Tugas 2.1

No.	File	Deskripsi Kelas
1	`apple-scab.JPG`	Apple Scab — bercak gelap tidak beraturan di permukaan daun
2	`grape-blackrot.JPG`	Grape Black Rot — lesi bulat besar berwarna coklat-hitam
3	`peach-bacterialspot.JPG`	Peach Bacterial Spot — bercak kecil tersebar, disertai klorosis
4	`soybean-healthy.JPG`	Soybean Healthy — daun sehat, venasi jelas, warna hijau uniform
5	`tomato-leafmold.JPG`	Tomato Leaf Mold — perubahan warna difus kekuningan

Setiap gambar diproses melalui tiga tahap preprocessing:
1. Konversi ke **grayscale** menggunakan `cv2.COLOR_BGR2GRAY`.
2. **Resize** ke ukuran tetap 256×256 piksel.
3. **Normalisasi** ke rentang $[0, 1]$ dengan membagi nilai piksel dengan 255.

2.1.2 Definisi Kernel

a) Low-pass: Gaussian 5×5 ($\sigma = 1$)

Dibangun dengan menempatkan nilai 1 pada pusat matriks nol 5×5, kemudian diterapkan `ndimage.gaussian_filter(sigma=1)`. Kernel ini berperan sebagai filter perata yang memberikan bobot lebih besar pada piksel terdekat.

b) High-pass: Laplacian 8-tetangga

$$L = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

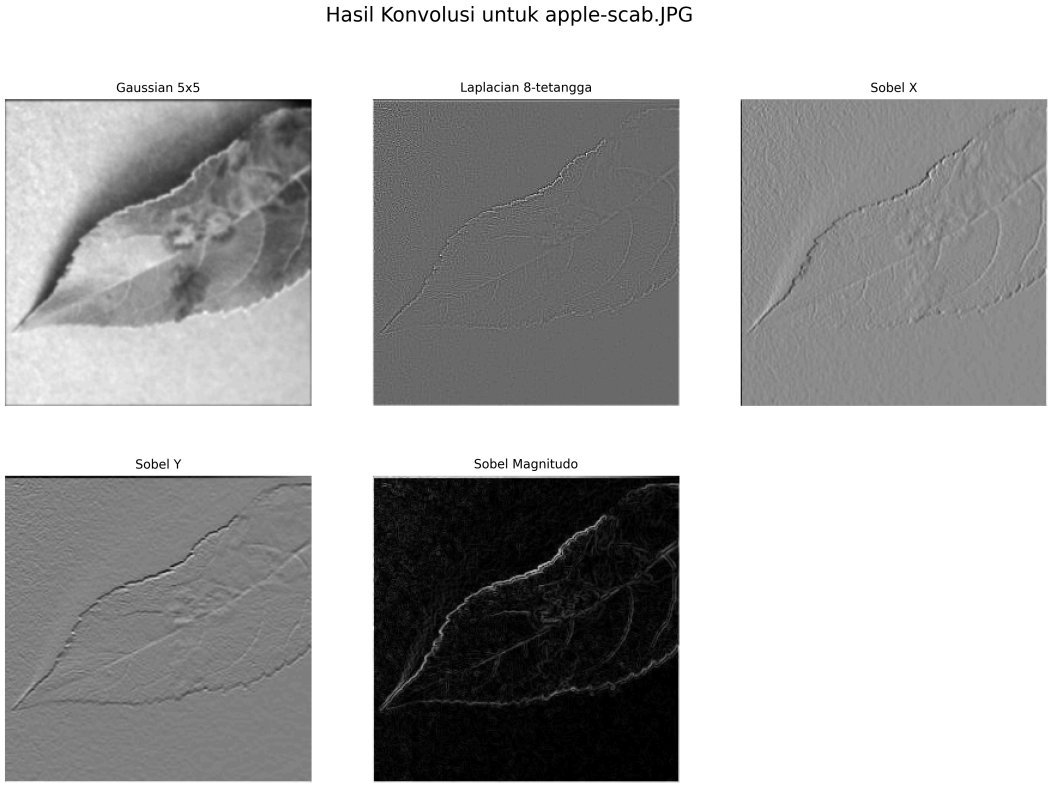
c) Deteksi Tepi: Sobel X, Sobel Y, dan Magnitudo

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad |G| = \sqrt{G_x^2 + G_y^2}$$

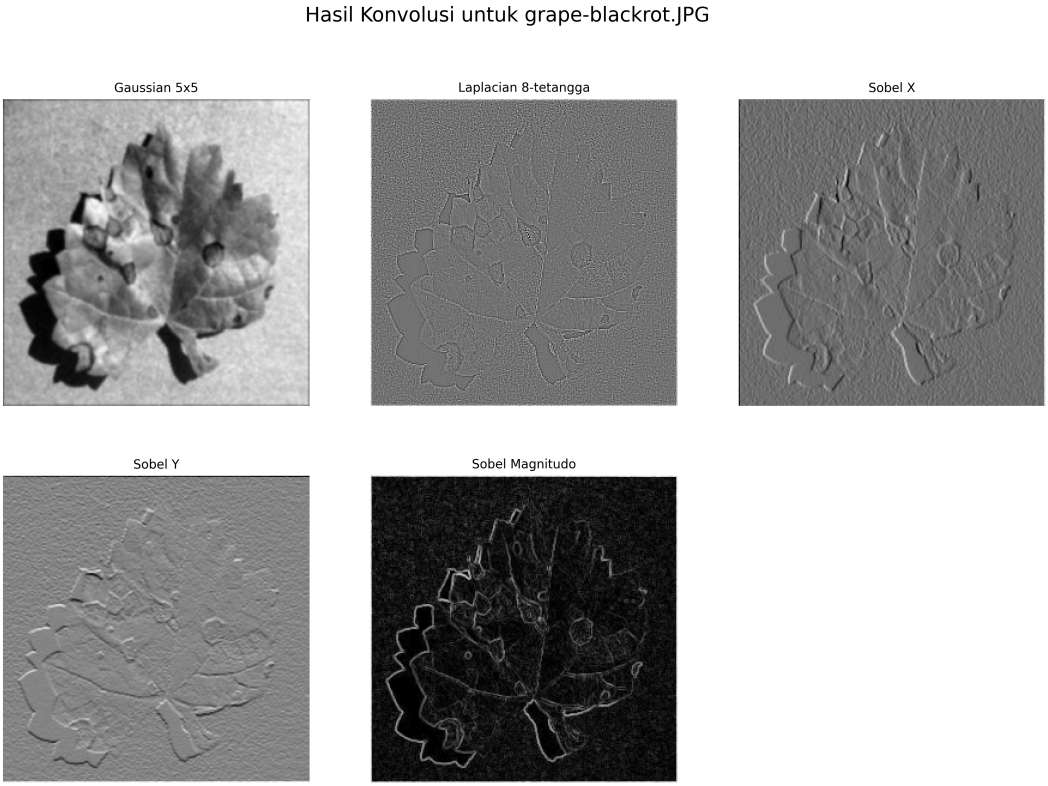
Semua konvolusi dilakukan dengan `scipy.signal.convolve2d` menggunakan `mode='same'`.

2.1.3 Hasil Visualisasi

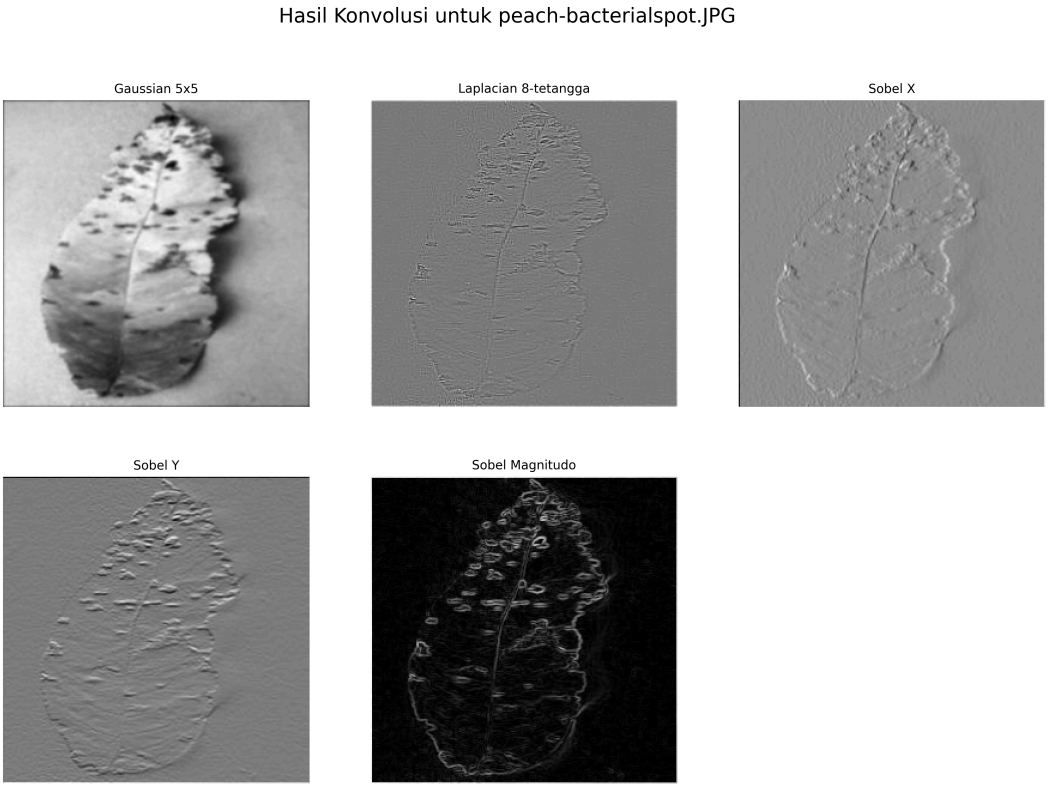
Untuk setiap citra ditampilkan subplot 2×3: (a) citra asli grayscale, (b) Gaussian, (c) Laplacian, (d) Sobel X, (e) Sobel Y, (f) Magnitudo Sobel.



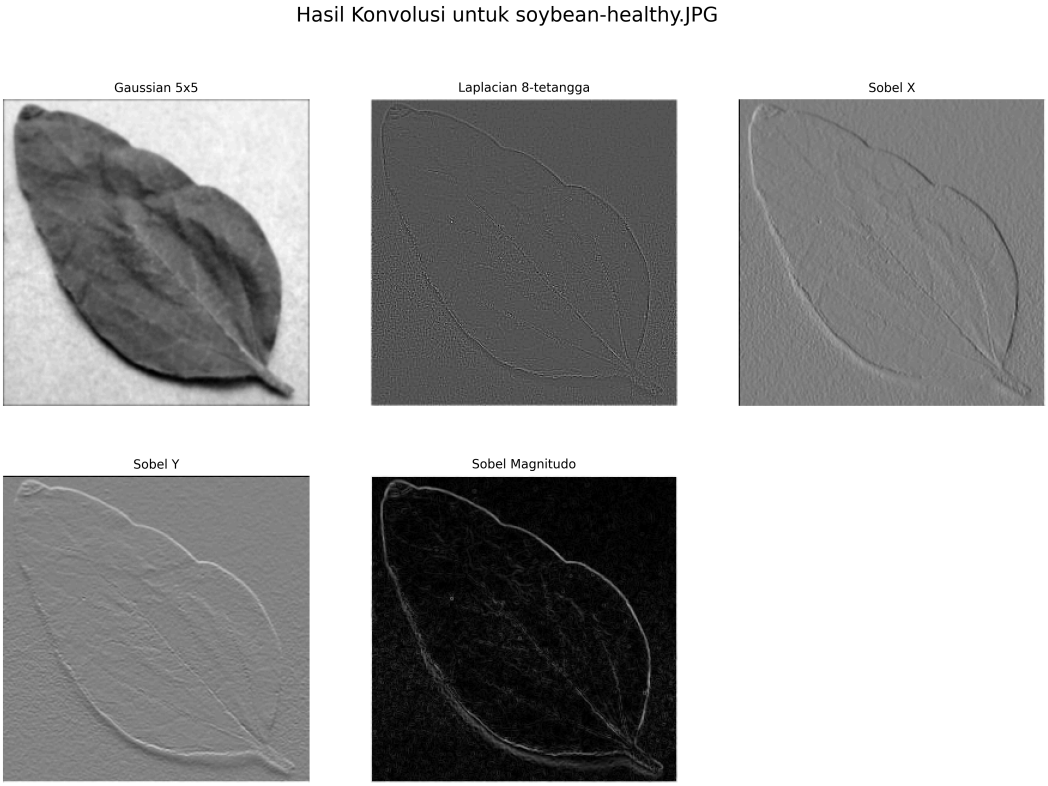
Gambar 1: Hasil konvolusi pada citra *apple-scab*



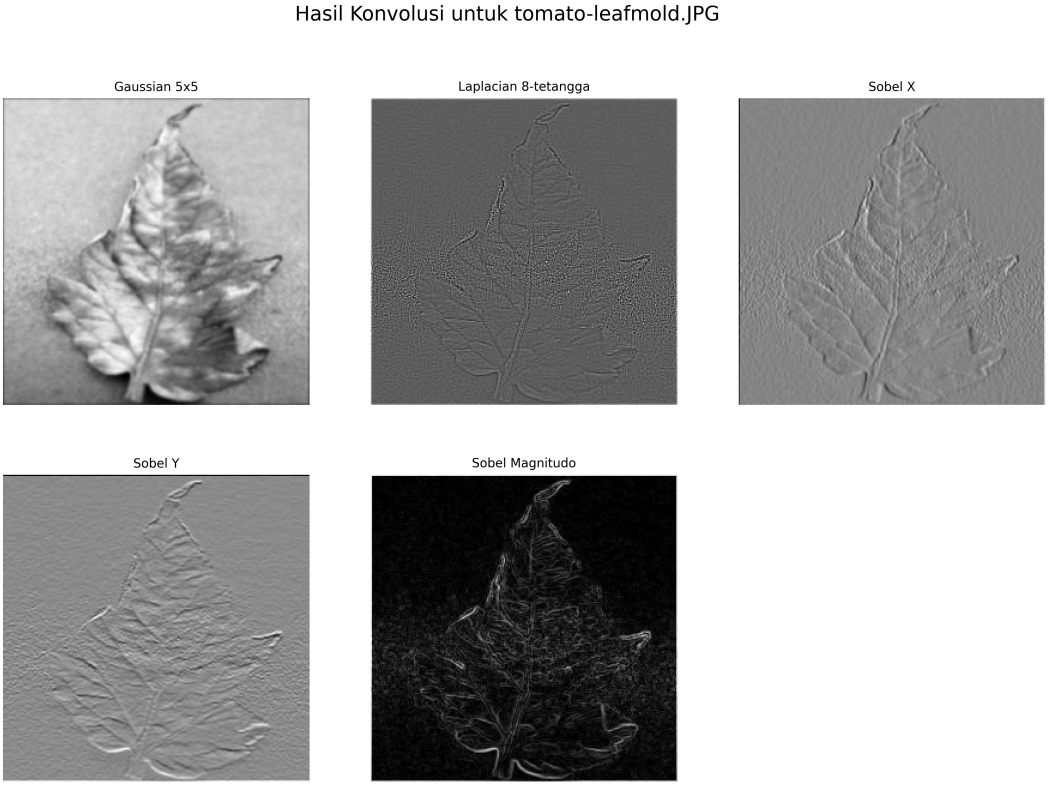
Gambar 2: Hasil konvolusi pada citra *grape-blackrot*



Gambar 3: Hasil konvolusi pada citra *peach-bacterialspot*



Gambar 4: Hasil konvolusi pada citra *soybean-healthy*



Gambar 5: Hasil konvolusi pada citra *tomato-leafmold*

2.1.4 Analisis

2.1.4.1 Fitur yang Ditonjolkan Masing-Masing Filter

Filter Gaussian menekan komponen frekuensi tinggi dengan merata-ratakan intensitas piksel berbobot jarak. Misalnya pada `grape-blackrot` menunjukkan bahwa transisi tajam antar-piksel berhasil diperhalus. Hasilnya secara visual tampak lebih "blur" dan tekstur permukaan daun yang kasar menjadi lebih halus.

Filter Laplacian merespons perubahan intensitas ke segala arah sekaligus sehingga menonjolkan seluruh struktur tepi dan tekstur lokal. Pada `grape-blackrot` di antara semua citra, yang mencerminkan kontras tajam antara lesi besar coklat-hitam dan jaringan daun hijau sekitarnya. Pada `peach-bacterialspot` terlihat lebih tajam. Pada area seragam seperti tengah helai daun sehat, gambar Laplacian hampir seragam gelap karena tidak ada perubahan intensitas signifikan.

Sobel X mendeteksi tepi vertikal (transisi ke arah horizontal), sedangkan **Sobel Y** mendeteksi tepi horizontal. **Magnitudo Sobel** menggabungkan keduanya dan merupakan representasi tepi yang paling lengkap dan robust terhadap arah. `grape-blackrot` memiliki magnitudo tertinggi karena menghasilkan banyak tepi kuat di kedua arah, sedangkan `soybean-healthy` memiliki magnitudo terendah karena tekstur yang lebih seragam.

2.1.4.2 Gaussian untuk Reduksi Noise pada Citra Daun

Filter Gaussian efektif menekan noise frekuensi tinggi yang berasal dari tekstur permukaan daun, artefak sensor, atau variasi pencahayaan. Pada `tomato-leafmold`, citra ini memiliki gradien intensitas yang tersebar halus (bukan bercak tajam), sehingga Gaussian berhasil meratakan transisi tersebut tanpa banyak kehilangan informasi.

Gaussian paling bermanfaat sebagai **preprocessing** sebelum deteksi tepi. Dengan terlebih dahulu memperhalus noise, respons Sobel/Laplacian berikutnya menjadi lebih bersih dan tidak terdistraksi oleh artefak piksel tunggal.

2.1.4.3 Laplacian/Sobel untuk Pengukuran Luas Daun dan Deteksi Lesi

Peta magnitudo Sobel secara langsung merepresentasikan lokasi dan kekuatan tepi pada citra. Pada `grape-blackrot`, lesi bulat besar tampak sebagai tepi yang jelas pada output magnitudo, karena terdapat transisi tajam.

2.1.4.4 Filter Paling Berguna untuk Sistem Deteksi Dini Penyakit

Untuk sistem deteksi dini penyakit berbasis perubahan tekstur daun, **kombinasi Gaussian + Sobel Magnitudo** adalah yang paling direkomendasikan:

1. **Gaussian** sebagai preprocessing: mengurangi noise sensor dan variasi tekstur permukaan yang tidak relevan.
2. **Sobel Magnitudo** sebagai ekstraktor fitur utama: mendeteksi distribusi batas lesi secara robust terhadap arah. Data menunjukkan `grape-blackrot` memiliki rata-rata magnitudo hampir dua kali lipat kelas lain, sehingga fitur ini dapat langsung digunakan sebagai indikator keparahan penyakit.

2.2 Tugas 2.2 – Perbandingan Penanganan Batas (Border Handling)

2.2.1 Pembuatan Citra Sintetis

Citra sintetis berukuran 6×6 dibuat dengan persegi putih 4×4 di tengah latar hitam:

$$S = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2.2.2 Implementasi dan Hasil Numerik

Tabel 2: Empat metode penanganan batas yang diimplementasikan

Metode	**Implementasi Python**	**Perilaku di Tepi**
Zero Padding	`signal.convolve2d(mode='same', boundary='fill')`	Piksel luar citra dianggap bernilai 0
Replikasi	`ndimage.convolve(mode='nearest')`	Piksel tepi direplikasi ke luar batas
Refleksi	`ndimage.convolve(mode='mirror')`	Nilai dicerminkan dari piksel dalam
Cropping	`signal.convolve2d(mode='valid')`	Output 4×4, hanya posisi kernel muat penuh

Hasil numerik **Zero Padding** (output 6×6):

$$R_{\text{zero}} = \begin{bmatrix} -1 & -1 & 0 & 0 & 1 & 1 \\ -3 & -3 & 0 & 0 & 3 & 3 \\ -4 & -4 & 0 & 0 & 4 & 4 \\ -4 & -4 & 0 & 0 & 4 & 4 \\ -3 & -3 & 0 & 0 & 3 & 3 \\ -1 & -1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Hasil numerik **Replikasi** (output 6×6):

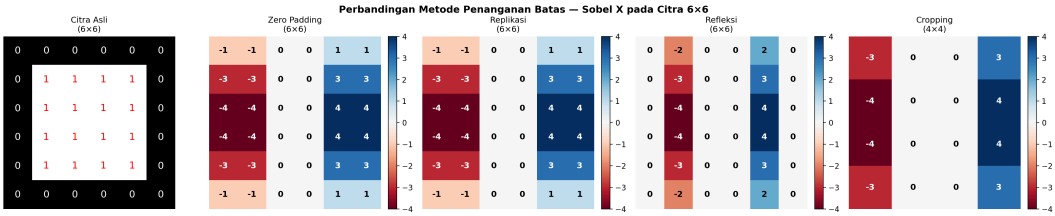
$$R_{\text{replicate}} = \begin{bmatrix} -1 & -1 & 0 & 0 & 1 & 1 \\ -3 & -3 & 0 & 0 & 3 & 3 \\ -4 & -4 & 0 & 0 & 4 & 4 \\ -4 & -4 & 0 & 0 & 4 & 4 \\ -3 & -3 & 0 & 0 & 3 & 3 \\ -1 & -1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Hasil numerik **Refleksi** (output 6×6):

$$R_{\text{mirror}} = \begin{bmatrix} 0 & -2 & 0 & 0 & 2 & 0 \\ 0 & -3 & 0 & 0 & 3 & 0 \\ 0 & -4 & 0 & 0 & 4 & 0 \\ 0 & -4 & 0 & 0 & 4 & 0 \\ 0 & -3 & 0 & 0 & 3 & 0 \\ 0 & -2 & 0 & 0 & 2 & 0 \end{bmatrix}$$

Hasil numerik **Cropping** (output 4×4):

$$R_{\text{crop}} = \begin{bmatrix} -3 & 0 & 0 & 3 \\ -4 & 0 & 0 & 4 \\ -4 & 0 & 0 & 4 \\ -3 & 0 & 0 & 3 \end{bmatrix}$$



Gambar 6: Perbandingan keempat metode penanganan batas pada citra sintetis 6×6

2.2.3 Analisis

Zero Padding Nilai di luar batas dianggap 0. Menghasilkan respons yang melemah di tepi dan sudut citra karena sebagian kernel bertemu nilai nol yang tidak representatif. Artefak berupa gradien palsu muncul di baris/kolom paling luar.

Replikasi Piksel paling tepi direplikasi ke luar. Untuk citra sintetis ini hasilnya identik dengan zero padding karena piksel tepi bernilai 0 — replikasi 0 sama dengan zero padding. Pada citra nyata (tepi bukan nol), replikasi lebih baik dari zero padding.

Refleksi Nilai dicerminkan dari dalam ke luar batas. Menghasilkan artefak paling sedikit karena nilai bayangan konsisten dengan konten aktual citra. Terlihat dari hasil: kolom paling tepi bernilai 0 (bukan ±1 seperti zero padding), menunjukkan tidak ada gradien palsu di batas.

Cropping Hanya menghitung posisi di mana kernel muat sepenuhnya sehingga output mengecil menjadi 4×4. Tidak ada artefak sama sekali, namun informasi di tepi citra hilang.

2.2.3.1 Rekomendasi untuk Aplikasi Pertanian

Dalam analisis citra daun pertanian, tepi daun sangat kritis karena banyak penyakit muncul pertama kali di tepi helai daun, dan pengukuran luas daun memerlukan deteksi kontur akurat hingga piksel paling tepi. Metode **Refleksi** (`mode='mirror'`) paling direkomendasikan: output berukuran sama dengan input, artefak tepi minimal, dan nilai padding konsisten secara spasial dengan konten aktual citra daun

— berbeda dengan zero padding yang memperkenalkan transisi palsu antara nol dan nilai piksel tepi.

2.3 Tugas 2.3 – Aplikasi Sederhana: Identifikasi Tekstur Penyakit

2.3.1 (a) Desain Kernel

Tiga kelas dipilih untuk dibedakan: **Apple Scab** (bercak gelap berbatas tegas), **Apple Rust** (bercak oranye dengan tekstur berbulu, lebih difus), dan **Apple Black Rot** (lesi gelap besar dengan tepi tidak beraturan).

Kernel yang dirancang adalah **Laplacian 4-tetangga** sebagai pendeteksi energi tekstur lokal:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Alasan pemilihan setiap angka:

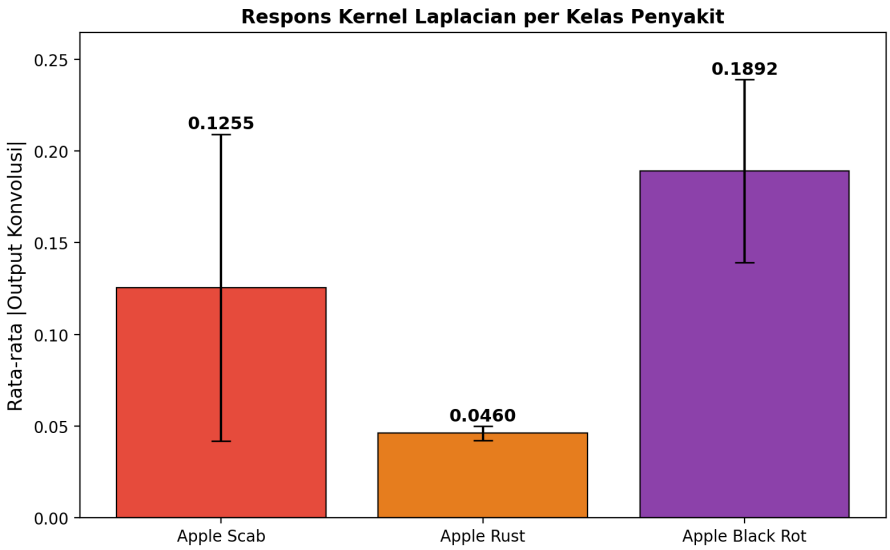
- Nilai pusat $+4$: mengangkat intensitas piksel pusat terhadap rata-rata tetangganya.
- Nilai tetangga -1 (atas/bawah/kiri/kanan): mengurangi kontribusi piksel sekitar secara aksial.
- Diagonal $= 0$: membuat kernel fokus pada struktur aksial (venasi, tepi bercak) dan tidak terlalu sensitif terhadap noise diagonal.
- Pada area **uniform** (intensitas konstan), jumlah perkalian menghasilkan sekitar 0. Pada area bertekstur atau berbercak, kernel menghasilkan nilai absolut yang tinggi.

2.3.2 (b) Hasil Rata-Rata Output Konvolusi per Kelas

Kernel diterapkan pada 5 sampel dari setiap kelas menggunakan `scipy.signal.convolve2d` dengan `mode='same'` dan `boundary='symm'`. Rata-rata nilai absolut output dirangkum pada tabel berikut:

Tabel 3: Rata-rata respons kernel Laplacian 4-tetangga per kelas

Kelas	**Rata-rata |Output|**	**Karakteristik Tekstur**
Apple Scab	0.1255	Bercak gelap berbatas tegas, kontras tinggi
Apple Rust	0.0460	Bercak difus berbulu, transisi intensitas halus
Apple Black Rot	0.1892	Lesi besar tidak beraturan, tepi kasar



Gambar 7: Bar chart perbandingan rata-rata |output konvolusi| ketiga kelas penyakit

2.3.3 (c) Evaluasi Kernel

Berdasarkan hasil di atas, kernel berhasil memisahkan ketiga kelas dengan pola yang konsisten dengan karakteristik visualnya. **Apple Rust** menghasilkan respons terendah $(0,0460)$ karena teksturnya yang difus dan transisi intensitas yang gradual, perubahan antar piksel kecil sehingga Laplacian tidak teraktivasi kuat. **Apple Scab** berada di tengah $(0,1255)$ dengan bercak gelap yang berbatas cukup tegas. **Apple Black Rot** menghasilkan respons tertinggi $(0,1892)$ karena lesinya yang besar dengan tepi kasar dan tidak beraturan, menghasilkan banyak transisi tajam yang direspons kuat oleh kernel.

Selisih antara kelas terendah dan tertinggi mencapai $0,143$, yang menunjukkan kernel ini **cukup efektif** sebagai pemisah kelas. Namun perlu dicatat bahwa standar deviasi Apple Scab cukup besar, yang mengindikasikan variasi tekstur antar sampel dalam kelas tersebut cukup tinggi.

2.3.4 (d) Diskusi: Konvolusi Fixed Kernel vs. Learned Kernels pada CNN

Konvolusi dengan kernel tetap seperti yang digunakan pada tugas ini memiliki kelebihan berupa interpretabilitas tinggi. Namun keterbatasannya nyata: satu kernel hanya menangkap satu jenis fitur, sehingga kelas yang memiliki karakteristik serupa (misalnya dua jenis bercak dengan ukuran berbeda) sulit dibedakan hanya dari satu nilai skalar rata-rata.

Learned kernels pada CNN bekerja secara berbeda. Layer konvolusi pertama CNN secara otomatis mempelajari puluhan filter yang secara fungsional menyerupai Sobel, Gaussian, dan Laplacian, namun dioptimalkan khusus untuk tugas klasifikasi yang diberikan. Layer lebih dalam kemudian menggabungkan fitur-fitur tingkat rendah tersebut menjadi representasi tingkat tinggi seperti bentuk lesi, distribusi bercak, dan pola warna. Untuk dataset sebesar New Plant Disease dengan

87.000 gambar dan 38 kelas, pendekatan CNN jauh lebih unggul karena mampu menangkap kompleksitas pola yang tidak dapat direpresentasikan oleh satu kernel tunggal.

Kesimpulannya, kernel manual berguna sebagai langkah eksplorasi awal dan preprocessing (seperti pengurangan noise dengan Gaussian sebelum ekstraksi fitur), sedangkan learned kernels pada CNN lebih cocok untuk sistem produksi yang membutuhkan akurasi tinggi pada banyak kelas sekaligus.